

THE OPEN UNIVERSITY OF SRI LANKA
DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING
BACHELOR OF SOFTWARE ENGINEERING (HONORS)
ACADEMIC YEAR 2022/23
EEI6373 PERFORMANCE MODELING

AN E-COMMERCE CLOUD SERVER SYSTEM



T.M.WANIGASINGHE | 722510838 | S22009999

Contents

1. System Description and Performance Goals.....	3
2. Modeling Approach and Assumptions.....	3
3. Data Description and Methodology.....	4
4. Simulation Scenarios and Setup.....	5
5. Detailed Analysis and Findings.....	5
6. Visualizations.....	7
7. Limitations and Future Work	9
8. References.....	10
9. Appendix	10

1. System Description and Performance Goals

System Description

This mini project evaluates the performance of an e-commerce cloud server system. An e-commerce system receives customer requests such as product searches, product views, shopping cart updates, and order requests. These requests are sent to a cloud server for processing.

Under normal traffic, the server can process requests with acceptable response times. However, during events such as flash sales and special promotions, the number of incoming requests can increase rapidly. When the arrival rate approaches the server's processing capacity, requests begin to wait in a queue. This increases response time and can create a performance bottleneck.

In this project, the e-commerce cloud server is modeled as a queueing system. Customer requests are treated as arrivals, and the cloud server acts as the service mechanism. The project compares a single-server system with a two-server system to study whether horizontal scaling can reduce congestion and improve performance.

Performance Goals

The main performance goals are:

- Reduce average response and waiting time.
- Reduce the number of requests waiting in the queue.
- Identify server congestion and bottlenecks.
- Study the effect of increasing request arrival rates.
- Measure server utilization and throughput.
- Compare single-server and multi-server configurations.
- Identify a suitable scaling method for high traffic.

2. Modeling Approach and Assumptions

Modeling Approach

Python simulation is also used to study individual request behaviour. The simulation uses an M/M/1 queue with an arrival rate of 90 requests per second and a service rate of 100 requests per second. A total of 1,000 requests are generated. The simulation records arrival time, service time, waiting time, and response time for each request.

Analytical M/M/1 calculations are used to compare system performance at different arrival rates of 40, 70, 90, and 98 requests per second. The M/M/2 model is evaluated analytically using the Erlang-C approach to study the effect of adding a second server.

For M/M/1:

$$\rho = \lambda / \mu$$

$$W = 1 / (\mu - \lambda)$$

$$Lq = \rho^2 / (1 - \rho)$$

- The service rate used in this project is - $\mu = 100$ requests/second

The M/M/2 model uses two parallel servers.

- At 90 requests/second - $\rho = 90 / (2 \times 100) = 0.45 = 45\%$
- Using the Erlang-C method, the average response time is approximately 12.54 ms.

Therefore, M/M/1 and M/M/2 can be compared to study the effect of horizontal scaling.

The following assumptions are used:

- Requests arrive randomly and independently.
- Request arrivals follow a Poisson process.
- Inter-arrival times follow an exponential distribution.
- Service times follow an exponential distribution.
- Requests use First-Come, First-Served (FCFS) processing.
- Queue capacity is unlimited.
- All servers have the same processing capacity.
- Network delay is not included.
- Database processing delay is not included.
- Load-balancing overhead is not included.

3. Data Description and Methodology

Real production data from an e-commerce cloud server were not available. Therefore, simulated data were generated using Python.

The simulation generates random inter-arrival times and service times using exponential distributions. The service rate is 100 requests/second, so the average service time is:

$$1 / 100 = 0.01 \text{ seconds} = 10 \text{ ms}$$

For the main simulation, 1,000 requests are generated. For each request, the following information is recorded:

- Request ID
- Arrival time
- Service time
- Service start time

- Waiting time
- Completion time
- Response time

Response time is calculated as $\text{Response Time} = \text{Completion Time} - \text{Arrival Time}$

The simulated data can be saved as a CSV file for further analysis. Analytical queueing results are also calculated and used to compare different traffic conditions.

4. Simulation Scenarios and Setup

The system is evaluated using different traffic conditions. The service rate of one server is fixed at 100 requests per second.

For the M/M/1 analytical model, four arrival rates are considered: 40, 70, 90, and 98 requests per second. These represent normal, moderate, high, and very high traffic conditions.

The Python request-level simulation is performed at 90 requests per second using 1,000 requests. This represents a high-load condition and is used to observe individual request behaviour.

Finally, an M/M/2 configuration is evaluated analytically using two servers, where each server has a service rate of 100 requests per second. The arrival rate is kept at 90 requests per second to compare the one-server and two-server configurations.

Scenario	Arrival Rate	Service Rate	Model	Description
Normal Load	40 req/s	100 req/s	M/M/1	Normal traffic
Moderate Load	70 req/s	100 req/s	M/M/1	Increased traffic
High Load	90 req/s	100 req/s	M/M/1	Peak traffic
Very High Load	98 req/s	100 req/s	M/M/1	Near capacity
Scaled System	90 req/s	100 req/s/server	M/M/2	Two-server system

Table 4.1 – Simulation Scenarios

The first four scenarios show how a single server behaves when traffic increases. The final scenario shows how adding a second server affects performance.

5. Detailed Analysis and Findings

M/M/1 Results

The 40, 70, 90, and 98 requests-per-second results are analytical M/M/1 results. The Python simulation is separately used at 90 requests per second to observe individual request behaviour.

Scenario	Arrival Rate	Utilization	Average Queue Size	Response Time
Normal Load	40 req/s	40%	0.27	16.67 ms
Moderate Load	70 req/s	70%	1.63	33.33 ms
High Load	90 req/s	90%	8.10	100 ms
Very High Load	98 req/s	98%	48.02	500 ms

Table 5.1 – M/M/1 Performance Results

- At 40 requests/second, the server has enough unused capacity. The average response time is 16.67 ms and the queue size is only 0.27 requests.
- At 70 requests/second, utilization increases to 70% and response time increases to 33.33 ms.
- At 90 requests/second, utilization reaches 90%. The response time increases to 100 ms, while the average queue size becomes 8.10 requests.
- At 98 requests/second, utilization reaches 98%. The response time increases sharply to 500 ms and the queue size increases to 48.02 requests.

These results show that response time does not increase linearly when utilization becomes close to 100%. A small increase in traffic can create a large increase in waiting time.

The Python simulation at 90 requests/second produced an average response time of 165.24 ms for the 1,000 simulated requests. This differs from the theoretical M/M/1 response time of 100 ms because the simulation uses randomly generated request arrivals and service times.

Throughput Analysis

The theoretical service capacity of the single server is 100 requests/second.

Since all tested arrival rates are below 100 requests/second, the system is theoretically stable. Therefore, the theoretical throughput is approximately equal to the arrival rate.

However, high throughput does not always mean good performance. For example, at 98 requests/second, the system can process many requests, but the average response time reaches 500 ms.

Therefore, throughput should be considered together with response time, queue size, and utilization.

M/M/1 vs M/M/2

At 90 requests/second:

Model	Servers	Utilization	Response Time
M/M/1	1	90%	100 ms
M/M/2	2	45%	12.54 ms

The response time is reduced by approximately 87.46% after adding a second server.

Key Findings

- Increasing arrival rates increases server utilization.
- High utilization causes larger queues and longer response times.
- A single server becomes a bottleneck during peak traffic.
- High throughput alone does not guarantee good performance.
- Adding a second server significantly improves response time.
- Horizontal scaling is useful for handling high traffic.

6. Visualizations

Three graphs are presented in this section.

Average Response Time

This graph shows that response time increases rapidly as the arrival rate approaches server capacity.

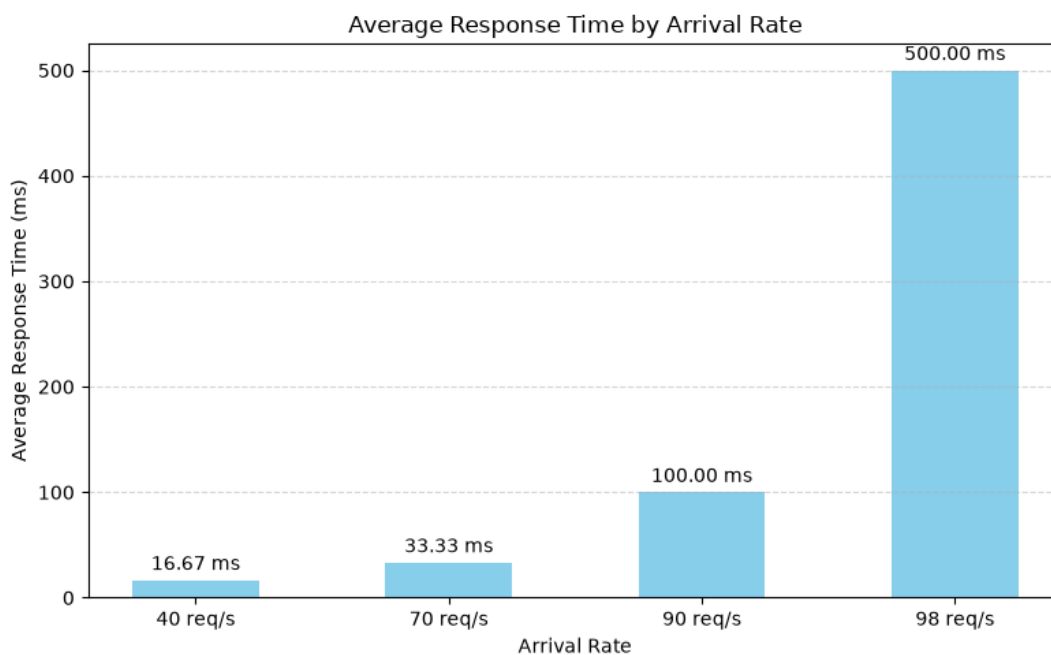


Figure 6.1 – Average Response Time by Arrival Rate

Average Queue Size

Figure 6.2 shows the average queue size at different server utilization levels. The queue size increases rapidly when server utilization becomes high.

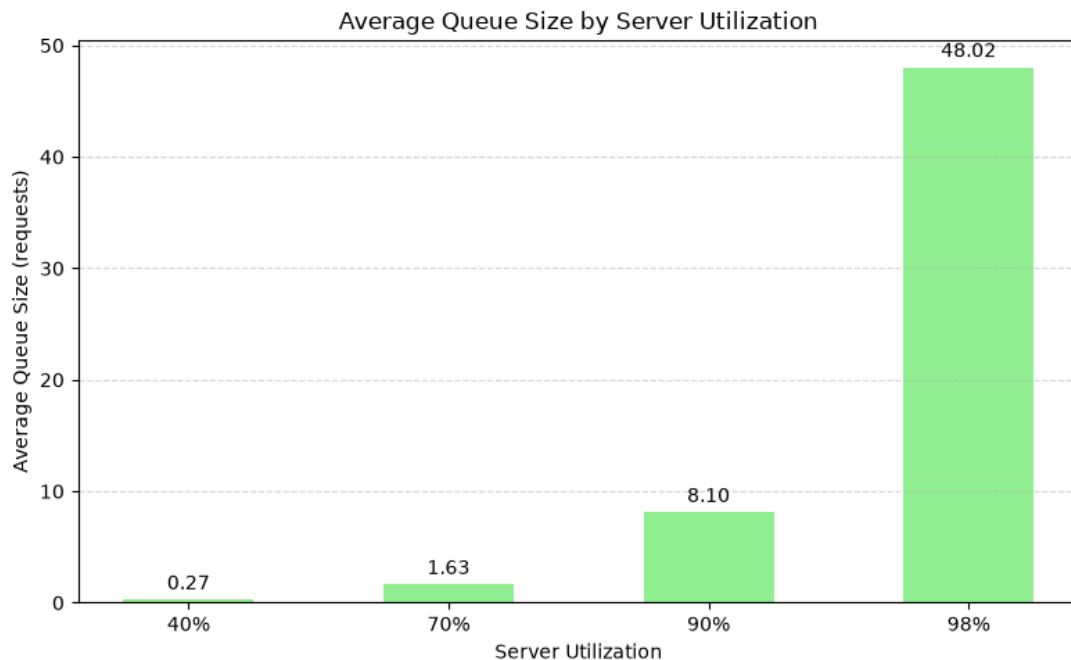


Figure 6.2 – Average Queue Size by Server Utilization

M/M/1 vs M/M/2

This graph compares the response times of one-server and two-server configurations at 90 requests/second.

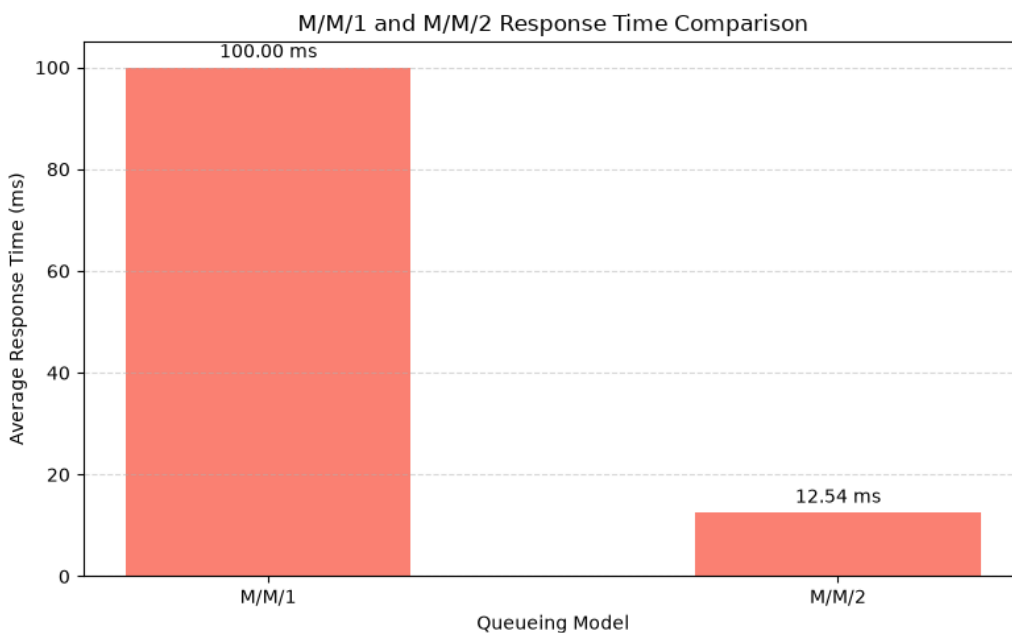


Figure 6.3 – M/M/1 and M/M/2 Response Time Comparison

These visualizations clearly show the performance impact of increasing traffic and the benefit of horizontal scaling.

7. Limitations and Future Work

Limitations

This project uses simulated data because real production data were not available. Real e-commerce traffic may not always follow a perfect Poisson process. Flash sales can create sudden and bursty traffic.

The model does not include network latency, database processing, caching delays, disk I/O, or load-balancing overhead. Service times are also assumed to follow an exponential distribution, while real customer requests may require different processing times.

The M/M/2 model assumes identical servers. The queue is also assumed to be unlimited, while real systems may reject or timeout requests when resources are exhausted.

Future Work

Future work can use real e-commerce server logs to make the model more realistic. Network and database delays can also be included.

The model can be extended to M/M/3 or larger multi-server systems. Automatic cloud scaling, Kubernetes Horizontal Pod Autoscaling, and different load-balancing methods can also be studied.

Future analysis can also include bursty traffic, server cost, and response-time measures such as P95 and P99.

8. References

- [1] L. Kleinrock, Queueing Systems, Volume I: Theory. New York, NY, USA: Wiley, 1975.
- [2] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, Fundamentals of Queueing Theory, 4th ed. Hoboken, NJ, USA: Wiley, 2008.
- [3] R. Jain, The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling. New York, NY, USA: Wiley, 1991.
- [4] A. M. Law, Simulation Modeling and Analysis, 5th ed. New York, NY, USA: McGraw-Hill, 2015.
- [5] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

9. Appendix

Git Hub Link - <https://github.com/thushaniwanigasinghe/E-commerce-Cloud-Server-Performance.git>

END